

Making networks safe and agile with formal methods and programming abstractions

For years, networks have been seen as hard to manage and hard to evolve. They are hard to manage because even small networks are complex, with multiple devices and protocols interacting to provide basic services, and human operators usually have few tools to help them reason about end-to-end behavior. They are hard to evolve because true innovation usually requires buy-in from vendors and standards bodies, and researchers and practitioners often do not have the stamina to wait the 3-4 years required to go from idea to fruition.

In recent years, however, research at the intersection of formal methods, programming languages, and networking has enabled researchers and practitioners to experiment with new architectures and abstractions. In a nutshell, this research enables operating on a different trade-off curve between agility (i.e., how quickly can you change a network) and safety (how confident can you be that the changes made are correct). These ideas have been developed in the context of software-defined networking (SDN), programmable data planes (P4, eBPF, etc.) as well as conventional networks (e.g., distributed routing protocols, fixed-function devices, etc.)

While there has been admirable progress over the past decade, many hard challenges remain—challenges that only approaches based on formal methods and programming languages can address. For agility, efforts like SDN, P4, eBPF, etc. provide the basic building blocks for programmable networks, but they largely leave open the question of what software architectures, languages, and APIs we should actually use to specify and implement real-world functionality. A lack of investment in this area risks: a return to ossification, fragmenting the community as proprietary abstractions get entrenched, and networks again become complex monoliths that are hard to program, stifling innovation. This concern is especially important in emerging areas like SmartNICs as well as network support for ML and AI, where hardware vendors are currently engaged in intense competition to dominate the marketplace.

For safety, experience has shown that even the most sophisticated tech companies (i.e., hyperscalers) struggled to provide the kind of reliability they need to support their businesses. There are growing efforts to apply ideas from formal methods and software engineering to networking – e.g., most cloud companies now have network verification teams (in some form) and there have also been startups in this space (Intentionet, Forward, VeriFlow, etc.) But it's still early days. Most of these tools only operate in certain niches, which limits their applicability and hinders transfer of ideas between researchers working on similar topics. To truly move the community forward, we need new approaches and the research community needs access to representative scenarios and benchmarks.

The rest of this section sketches four directions for future research in the area.

Modular design and reasoning

The global network is composed of many overlapping domains. These include not just traditional ISPs and enterprise networks but also virtual networks that span physical and (multiple) clouds, IoT networks, telco networks with different standards, etc. Despite this compositional structure, the perspective of almost all current tools for network synthesis and verification tends to be that of a single (homogeneous) domain. This makes verification and synthesis an all-or-nothing proposition—it does not admit fast local checking and enforcement of the local rules that lead to strong end-to-end properties. As developers update and maintain components of a network, they get no feedback about the effects of changes on neighboring components.

A failure to recognize the modular structure also makes it hard to scale verification and develop robust specifications. A "domain" or "virtual network" has its own purpose and abstractions, as well as its own network design, including namespace, topology, routing, and session protocols. Only with recognition of these specialized concepts is it possible to write strong specifications of network services as experienced by their users. Further, the only way to ensure that lower levels of the network do not interfere unnecessarily with applications is to properly separate their concerns.

We should make modularity and composition first-order concerns in the next generation of synthesis and verification tools. This research will require defining interfaces between domains and, while reasoning about one domain, handling uncertainty in the behavior of other domains. What information should such interfaces contain? How should it be expressed? Modular and compositional reasoning can facilitate proofs of end-to-end reachability, security, reliability, and performance guarantees across heterogeneous domains or systems and formalize how those domains depend on each other. In the control plane, they can help deliver greater reliability, fault isolation, and a means to support richer policies and algorithms.

Along with leveraging the modular structure in specification and verification, we should be encouraging modular structure in network implementations. Even without verification, it can lead to simplification, reuse, and increased reliability. We should also investigate how modularity can be exploited to solve practical problems experienced by network operators and application developers, such as untangling the "tangle of tunnels" in IP networks.

Verifying modern control planes

Network control planes used to (primarily) be the configuration of distributed routing protocols, and they used to be self-contained with no external dependencies. But with the rise of SDN and network virtualization, the network control planes have gotten richer and depend on many other systems (e.g., for performance and demand data) to determine how the network forwards traffic. In some cases, these dependencies are circular as well, where a distributed database depends on the network, and the network depends on the database service. Such control planes, while highly flexible, can pose a serious threat to network reliability, as was demonstrated by the global Facebook outage in Oct 2021 that lasted for over seven hours and other large, cascading failures.

Most current verification technology has not targeted such modern control planes. Doing so effectively will require carefully formalizing interfaces between components and their interactions. The modular reasoning abilities mentioned earlier are a prerequisite here. Once we have the dependencies formalized, we can verify that no combination of K failures can cascade to a system-wide outage. We can also verify that the core control plane, which directly interacts with the network devices, is statically stable – that is, it can function (in degraded mode) even if all external dependencies are missing and it can recover from disasters.

Accessible network testing

While network verification has had undeniable success inside the largest of networks, such as those of hyperscalers, its penetration beyond them is still fairly limited. We think this state of affairs is down to the difficulty of using the current technologies for most network engineers. This difficulty stems from a host of factors, including (1) the lack of precise specifications that verification tools need, (2) mismatch in user expertise (e.g., network engineers cannot be expected to write logic queries), and (3) verification tools being conceptually different from traditional tools such as traceroute and ping (e.g., reasoning about large sets of packets instead of concrete packets). To make the vast majority of networks more reliable, we need to build tools that are easy to use for 95

We should take inspiration from the software domain. Here, while verification is still not commonplace, many capabilities to improve the reliability of software have become commonplace over the last two decades. These include type-safe languages such as Rust and Java, test frameworks such as junit, and programming environments that make it easy to employ such capabilities. We believe that networking can take a similar journey, where we turn focus on the accessibility of automatic testing instead of focusing solely on verification technologies that can provide strong safety guarantees. Example explorations: What does a type system for networks look like? How can we develop a test framework for networks that enables both proactive (before changes are pushed to the network) testing and continuous monitoring?

Thriving ecosystem of networking languages

A language-oriented approach to designing networks has many advantages. Languages with precise semantics as well as strong abstraction and composition features facilitate communication, reuse, and reorganization of ideas and algorithms. These ideas may be shared between developers and organizations, and accelerate network development. P4 is a good example, and it has enabled a broad range of hardware and application innovation. But there is significant unexplored territory, not just for languages that describe packet forwarding inside a switch but other aspects of networks such as the control plane, network-wide forwarding, middleboxes, access control, and layer7 networks such as those built using service meshes.

The research agenda here goes beyond developing surface language for different tasks. It includes developing one or more intermediate languages (ILs) that provide a common representation for multiple surface languages, in the same way, LLVM provides a platform for compiling and analyzing many general-purpose languages. Such ILs will provide a common basis for verification and optimization, without needing to do this work for each surface language.

Further, to facilitate verification and synthesis, it is usually helpful for languages to be designed with these goals in mind from the get-go. Languages facilitate the verification process through “correct by construction design”, which eliminates failure modes. Linguistic features such as embedded assertions, constraints, types, proofs, and symbolic values allow programmers to write, and more importantly, maintain proofs as they construct and maintain, synthesize, or re-synthesize programs. Equally important are the development environments in which programmers operate. Such environments can facilitate code construction and analysis of all kinds, as well as maintain proof state.

Keep software-defined networking open and evolving

By pursuing the programming language approach demonstrated by the foregoing software engineering and formal methods projects, we take a step forward in modernizing networking and basing it on disciplined software. The

resulting software infrastructure, if it truly retains the vitality of networking, will surpass any software we already know. Networking needs to remain open to multiple stakeholders, welcoming small newcomers, enabling partial deployment of new solutions, maintaining incentives for collaboration and competition, and gracefully evolving with unanticipated requirements that are bound to arise rapidly at all locations (layer, domain, aspect), while operating continuously and reliably, with a lifetime measured not by weeks or months but by years and decades. Even when considered at a fixed 'snapshot,' networking as a program means more than processing the object of located packets in a prefixed scenario. Instead, it needs to support, as object data of the language, a wider range of entities such as network events (arbitrary link failures), planned (configuration and policy) updates, network split/merge, to name a few.

These challenges, while not readily fitting into the stand-alone and monolithic software architecture often presupposed by 'classic' programming language techniques, echo similar problems extensively studied—albeit on a smaller scale and with less exciting scenarios—in neighboring disciplines such as databases, knowledge representation, and (logic-based) artificial intelligence. Databases form a long-lasting infrastructure designed for permanent evolvability. The universal SQL/datalog interface proves expressive for existing queries and any foreseeable future ones, mediating between multiple users under distinct policies. Sophisticated semantics-based query transformations provide systematic support for integrating new (heterogeneous) data sources on the fly. Finally, datalog and prolog are celebrated for meta-programming, treating other programs as object data of the language. This makes them an attractive alternative to encode the complex networking environment, where events and updates (programs) are to be manipulated as first principle data. We thus call for more investigation into a multitude of programming language approaches, combining efforts from classic software engineering, formal methods, data and knowledge representation, artificial intelligence, and more, as a means to advance the programming interface for future networking.